

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

Лабораторная работа 4

По дисциплине Объектно-ориентированное программирование

Тема работы Алгоритмы на графах

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа К3221

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникацион-
ных системах

Обучающийся

(дата)

(подпись)

Сакулин И.М.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Иванов С.Е.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 ПОИСК В ГЛУБИНУ И ШИРИНУ	4
1.1 Структура класса Graph	4
1.2 Поиск в глубину (DFS)	6
1.3 Поиск в ширину (BFS)	6
1.4 Пример использования	7
2 АЛГОРИТМ ДЕЙКСТРЫ.....	10
2.1 Реализация алгоритма Дейкстры	10
2.2 Пример использования	12
3 АЛГОРИТМ КРУСКАЛА.....	13
3.1 Класс Edge.....	13
3.2 Класс UnorderedGraph.....	15
3.3 Пример использования	16
4 КОНТРОЛЬНЫЕ ВОПРОСЫ	20
ЗАКЛЮЧЕНИЕ	21

ВВЕДЕНИЕ

В отчёте представлено решение лабораторной работы. В тексте работы представлены результаты реализации алгоритмов и ответы на контрольные вопросы.

Цель — научиться реализовывать алгоритмы на графах. Реализовать средствами ООП поиск в глубину и ширину на графе. Реализовать средствами ООП алгоритм нахождения кратчайших путей (Алгоритм Дейкстры). Реализовать алгоритм Крускала.

Задание на лабораторную работу

1. Реализовать средствами ООП поиск в глубину и ширину на графе. Выполнить расчет примера.
2. Реализовать средствами ООП алгоритм нахождения кратчайших путей из одного источника (Алгоритм Дейкстры). Выполнить расчет примера.
3. Найти минимальный остов неориентированного взвешенного графа (Алгоритм Крускала), представленного ниже.

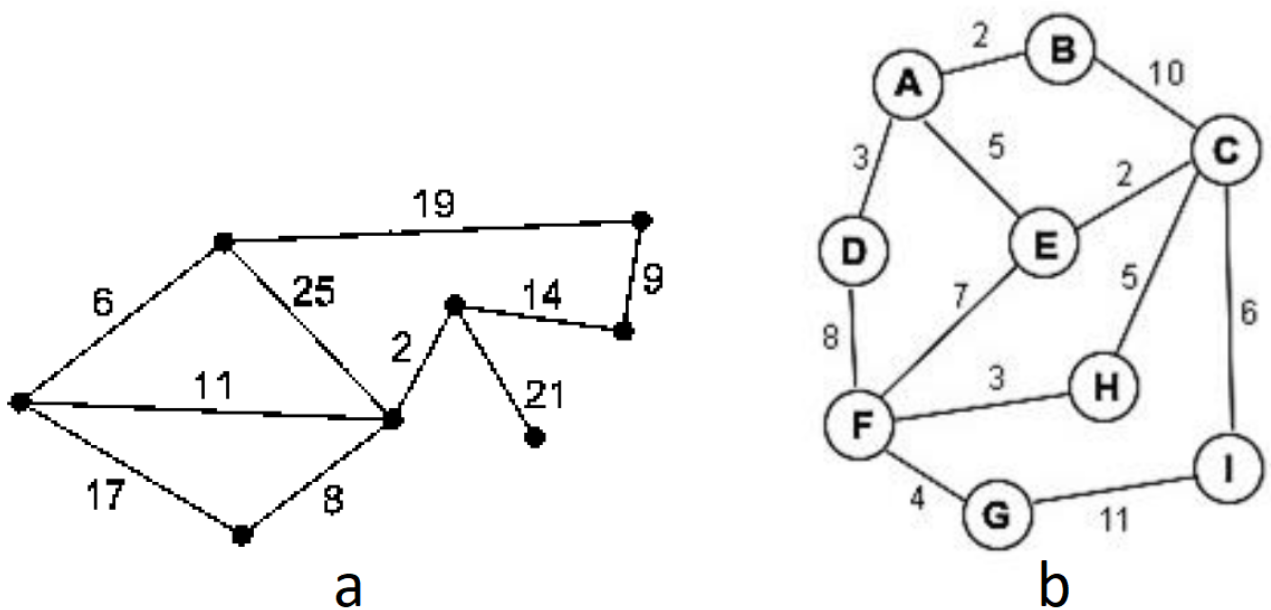


Рисунок 1 — Примеры графов

1 ПОИСК В ГЛУБИНУ И ШИРИНУ

1.1 Структура класса Graph

Класс `Graph` представляет граф с матрицей смежности. В нём определены поля для хранения размера графа, самой матрицы, границ генерации весов, а также вспомогательные структуры для обходов и алгоритма Дейкстры. Конструкторы позволяют создать граф на основе готовой матрицы или сгенерировать случайный граф с заданными параметрами.

```
1  // Поля и конструкторы класса Graph
2  public class Graph
3  {
4      protected static Random rand = new Random();
5      public int n { get; protected set; }
6      public int[] [] g;
7      private int min_val = 0, max_val = int.MaxValue;
8
9      public List<int> used = new List<int>();
10     public Queue<int> queue = new Queue<int>();
11
12     public bool[] viewed;
13     public int[] len;
14     public int[] next;
15
16     // Конструктор на основе готовой матрицы
17     public Graph(int[] [] gr)
18     {
19         n = gr.Count();
20         g = gr;
21     }
22
23     // Защищённый конструктор для выделения памяти
24     protected Graph(int size)
25     {
26         n = size;
27         g = new int[n] [];
28         for (int i = 0; i < n; i++) g[i] = new int[n];
29     }
```

```

30
31 // Публичный конструктор с генерацией случайного графа
32 public Graph(int size, bool oriented, int min_value = 0, int max_value =
    ↪ int.MaxValue - 1) : this(size)
33 {
34     min_val = min_value;
35     max_val = max_value;
36
37     if (oriented)
38     {
39         for (int i = 0; i < n; i++)
40         {
41             for (int j = 0; j < n; j++)
42             {
43                 if (i == j) continue;
44                 int r = rand.Next(0, 2) == 1 ? rand.Next(min_value,
    ↪ max_value + 1) : 0;
45                 g[i][j] = r;
46             }
47         }
48     }
49     else
50     {
51         for (int i = 0; i < n; i++)
52         {
53             for (int j = i + 1; j < n; j++)
54             {
55                 int r = rand.Next(0, 2) == 1 ? rand.Next(min_value,
    ↪ max_value + 1) : 0;
56                 g[i][j] = r;
57                 g[j][i] = r;
58             }
59         }
60     }
61 }
62 }

```

Рисунок 2 — Поля и конструкторы класса Graph

1.2 Поиск в глубину (DFS)

Метод DFS реализует поиск в глубину с использованием рекурсии. Он выводит текущую вершину и список посещённых, после чего рекурсивно вызывает себя для всех непосещённых смежных вершин.

```
1  // Метод DFS
2  public void DFS(int i, bool first = true)
3  {
4      if (first)
5      {
6          Console.WriteLine("\nDFS");
7          used = new List<int>();
8      }
9
10     Console.WriteLine(" Node {0}", i);
11     Console.WriteLine("Used: [{0}]", string.Join(", ", used));
12
13     used.Add(i);
14     for (int j = 0; j < n; j++)
15     {
16         if (g[i][j] == 1 && !used.Contains(j))
17         {
18             DFS(j, false);
19         }
20     }
21 }
```

Рисунок 3 — Реализация поиска в глубину

1.3 Поиск в ширину (BFS)

Метод BFS использует очередь для обхода в ширину. В начале в очередь помещаются все непосещённые соседи текущей вершины, затем рекурсивно обрабатывается следующий элемент из очереди.

```
1 // Метод BFS
2 public void BFS(int i, bool first = true)
3 {
4     if (first)
5     {
6         Console.WriteLine("\nBFS");
7         used = new List<int>();
8         queue = new Queue<int>();
9     }
10
11     Console.WriteLine(" Node {0}", i);
12     Console.WriteLine("Used: [{0}]", string.Join(", ", used));
13     Console.WriteLine("Queue: [{0}]", string.Join(", ", queue));
14
15     used.Add(i);
16     for (int j = 0; j < n; j++)
17     {
18         if (g[i][j] == 1 && !used.Contains(j) && !queue.Contains(j))
19         {
20             queue.Enqueue(j);
21         }
22     }
23
24     if (queue.Count() != 0) BFS(queue.Dequeue(), false);
25 }
```

Рисунок 4 — Реализация поиска в ширину

1.4 Пример использования

Для демонстрации работы создаётся неориентированный невзвешенный граф с шестью вершинами (параметры `false`, `1`, `1` задают отсутствие ориентации и единичные веса). Вызываются методы `DFS` и `BFS` от нулевой вершины.

```
1 // Фрагмент Program.cs
2 Graph g1 = new Graph(6, false, 1, 1);
3 g1.Print();
4 g1.DFS(0);
5 g1.BFS(0);
```

Рисунок 5 — Код примера для DFS и BFS

На рисунке 6 представлен результат поиска в глубину и в ширину.


```
Microsoft Visual Studio Debug Console

Graph
Node 0 : [000100]
Node 1 : [000001]
Node 2 : [000110]
Node 3 : [101011]
Node 4 : [001101]
Node 5 : [010110]

DFS
Node 0
Used: []
Node 3
Used: [0]
Node 2
Used: [0, 3]
Node 4
Used: [0, 3, 2]
Node 5
Used: [0, 3, 2, 4]
Node 1
Used: [0, 3, 2, 4, 5]

BFS
Node 0
Used: []
Queue: []
Node 3
Used: [0]
Queue: []
Node 2
Used: [0, 3]
Queue: [4, 5]
Node 4
Used: [0, 3, 2]
Queue: [5]
Node 5
Used: [0, 3, 2, 4]
Queue: []
Node 1
Used: [0, 3, 2, 4, 5]
Queue: []
```

Рисунок 6 — Результат поиска в глубину и ширину

2 АЛГОРИТМ ДЕЙКСТРЫ

2.1 Реализация алгоритма Дейкстры

Метод `Dejkstra` находит кратчайшие пути от заданной вершины. Он использует массивы `len` (текущие расстояния), `viewed` (обработанные вершины) и `next` (предыдущая вершина на пути). На каждом шаге выбирается необработанная вершина с минимальным расстоянием, и производится релаксация рёбер из неё. После завершения вызывается `PrintPaths` для вывода результатов.

```
1  // Реализация алгоритма Дейкстры
2  public void Dejkstra(int i, bool first = true)
3  {
4      if (first)
5      {
6          if (min_val < 0) throw new Exception("Negative weighted edges not
           ↳ allowed");
7          viewed = new bool[n];
8          len = new int[n];
9          next = new int[n];
10         for (int j = 0; j < n; j++)
11         {
12             viewed[j] = false;
13             len[j] = int.MaxValue;
14             next[j] = -1;
15         }
16         len[i] = 0;
17     }
18     viewed[i] = true;
19     int minDist = int.MaxValue;
20     int minVertex = -1;
21     for (int j = 0; j < n; j++)
22     {
23         if (g[i][j] != 0 && !viewed[j] && len[i] != int.MaxValue)
24         {
25             int newLen = len[i] + g[i][j];
26             if (newLen < len[j])
```

```

27         {
28             len[j] = newLen;
29             next[j] = i;
30         }
31     }
32     if (!viewed[j] && len[j] < minDist)
33     {
34         minDist = len[j];
35         minVertex = j;
36     }
37 }
38 if (minVertex != -1) Dejkstra(minVertex, false);
39 if (first) PrintPaths(i);
40 }
41
42 public void PrintPaths(int i)
43 {
44     Console.WriteLine("\n");
45     for (int j = 1; j < n; j++)
46     {
47         if (i == j) continue;
48         Console.WriteLine("Path to {1} from {0} (len={2}):", 0, j, (len[j] ==
49             ↪ int.MaxValue) ? "inf" : len[j].ToString());
50         int z = j;
51         while (z != 0 && z != -1)
52         {
53             Console.Write(z + " <- ");
54             z = next[z];
55         }
56         if (z == -1) Console.WriteLine("no way");
57         else Console.WriteLine(z);
58     }
59 }

```

Рисунок 7 — Методы Dejkstra и PrintPaths

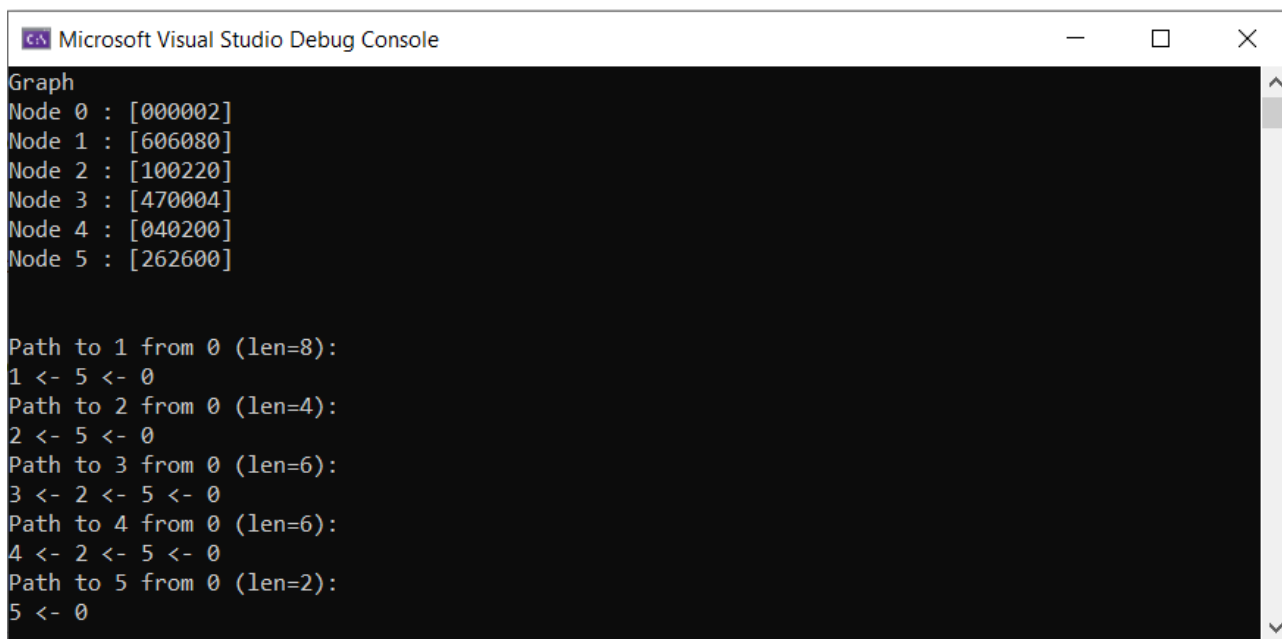
2.2 Пример использования

Для проверки создаётся ориентированный взвешенный граф с шестью вершинами (веса от 1 до 9). Запускается алгоритм от вершины 0.

```
1 // Фрагмент Program.cs
2 Graph g2 = new Graph(6, true, 1, 9);
3 g2.Print();
4 g2.Dejkstra(0);
```

Рисунок 8 — Пример использования алгоритма Дейкстры

На рисунке 9 представлен результат работы алгоритма.



```
Microsoft Visual Studio Debug Console

Graph
Node 0 : [000002]
Node 1 : [606080]
Node 2 : [100220]
Node 3 : [470004]
Node 4 : [040200]
Node 5 : [262600]

Path to 1 from 0 (len=8):
1 <- 5 <- 0
Path to 2 from 0 (len=4):
2 <- 5 <- 0
Path to 3 from 0 (len=6):
3 <- 2 <- 5 <- 0
Path to 4 from 0 (len=6):
4 <- 2 <- 5 <- 0
Path to 5 from 0 (len=2):
5 <- 0
```

Рисунок 9 — Результат работы алгоритма Дейкстры

3 АЛГОРИТМ КРУСКАЛА

3.1 Класс Edge

Класс `Edge` представляет ребро графа с полями `I`, `J` (вершины) и `Weight` (вес). Метод `Check` управляет компонентами связности в процессе построения остова. Он добавляет ребро в компоненту или объединяет компоненты, возвращая `true`, если ребро не создаёт цикл.

```
1  // Класс Edge
2  using System.Collections.Generic;
3
4  namespace Graphs
5  {
6      public class Edge
7      {
8          public int I { get; set; }
9          public int J { get; set; }
10         public int Weight { get; set; }
11
12         public Edge(int i, int j, int weight)
13         {
14             I = i;
15             J = j;
16             Weight = weight;
17         }
18         public override string ToString()
19         {
20             return "Edge (" + I + ", " + J + "): weight = " + Weight;
21         }
22         public bool Check(List<HashSet<int>> components)
23         {
24             HashSet<int> compI = null;
25             HashSet<int> compJ = null;
26             foreach (var comp in components)
27             {
28                 if (comp.Contains(I)) compI = comp;
29                 if (comp.Contains(J)) compJ = comp;
30             }
```

```

31      // Случай 1: обе вершины ещё не входят ни в одну компоненту
32      if (compI == null && compJ == null)
33      {
34          components.Add(new HashSet<int> { I, J });
35          return true;
36      }
37      // Случай 2: только I уже в какой-то компоненте
38      else if (compI != null && compJ == null)
39      {
40          compI.Add(J);
41          return true;
42      }
43      // Случай 3: только J уже в компоненте
44      else if (compI == null && compJ != null)
45      {
46          compJ.Add(I);
47          return true;
48      }
49      // Случай 4: обе вершины уже в компонентах
50      else // compI != null && compJ != null
51      {
52          if (compI == compJ)
53          {
54              // Ребро замыкает цикл
55              return false;
56          }
57          else
58          {
59              // Объединяем две разные компоненты
60              compI.UnionWith(compJ);
61              components.Remove(compJ);
62              return true;
63          }
64      }
65  }
66 }
67 }

```

Рисунок 10 — Класс Edge

3.2 Класс UnorderedGraph

Класс `UnorderedGraph` наследует `Graph` и предназначен для неориентированных взвешенных графов. В конструкторе на основе матрицы смежности формируется список рёбер `reb`. Метод `Kruskal` сортирует рёбра по весу, затем последовательно добавляет их в остов, используя метод `Check` для управления компонентами.

```
1  // Класс UnorderedGraph и метод Kruskal
2  using System;
3  using System.Collections.Generic;
4  using System.ComponentModel;
5
6  namespace Graphs
7  {
8      public class UnorderedGraph : Graph
9      {
10         public List<Edge> reb = new List<Edge>();
11
12         public UnorderedGraph(int size, int min_value = 0, int max_value =
13             → int.MaxValue - 1)
14             : base(size, false, min_value, max_value)
15         {
16             for (int i = 0; i < n; i++)
17             {
18                 for (int j = i + 1; j < n; j++)
19                 {
20                     if (g[i][j] != 0)
21                     {
22                         Edge edge = new Edge(i, j, g[i][j]);
23                         reb.Add(edge);
24                     }
25                 }
26             }
27
28             public void Kruskal()
29             {
```

```

30         reb.Sort((a, b) => a.Weight.CompareTo(b.Weight));
31
32         List<Edge> mst = new List<Edge>();
33         List<HashSet<int>> components = new List<HashSet<int>>();
34
35         foreach (Edge edge in reb)
36         {
37             if (edge.Check(components))
38             {
39                 mst.Add(edge);
40             }
41         }
42
43         Console.WriteLine("\nMinimum Spanning Tree (Kruskal):");
44         int totalWeight = 0;
45         foreach (Edge edge in mst)
46         {
47             Console.WriteLine(edge);
48             totalWeight += edge.Weight;
49         }
50         Console.WriteLine($"Total weight: {totalWeight}");
51     }
52 }
53 }

```

Рисунок 11 — Класс UnorderedGraph

3.3 Пример использования

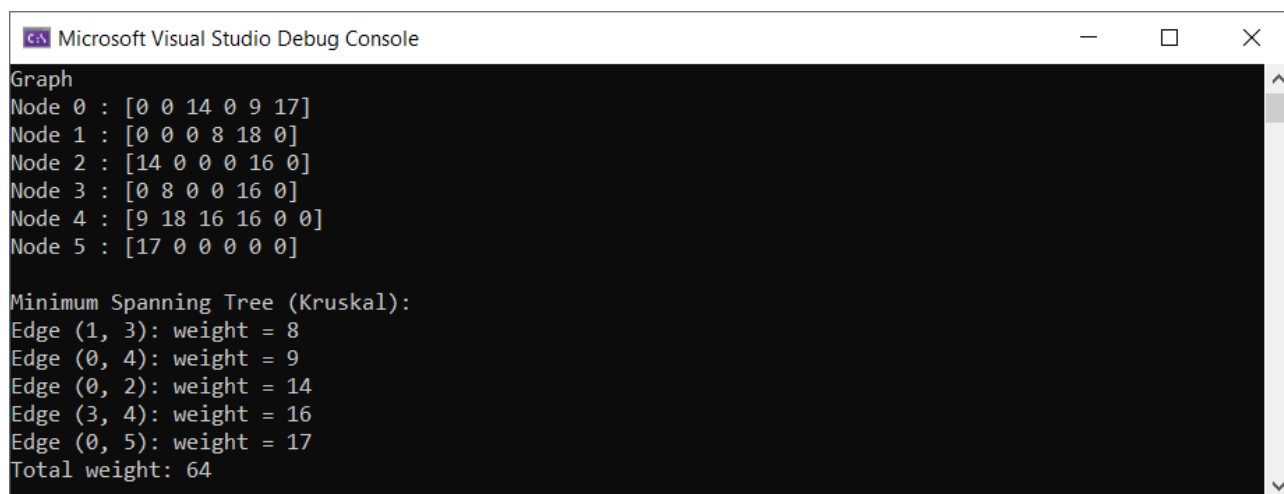
Создаётся неориентированный взвешенный граф с шестью вершинами (веса от 1 до 21). Вызывается метод `Kruskal`, который выводит рёбра минимального остова и их суммарный вес.

```

1  // Фрагмент Program.cs
2  UnorderedGraph g3 = new UnorderedGraph(6, 1, 21);
3  g3.Print();
4  g3.Kruskal();

```

Рисунок 12 — Пример использования алгоритма Крускала
На рисунке 13 представлен результат выполнения программы.



```
Microsoft Visual Studio Debug Console

Graph
Node 0 : [0 0 14 0 9 17]
Node 1 : [0 0 0 8 18 0]
Node 2 : [14 0 0 0 16 0]
Node 3 : [0 8 0 0 16 0]
Node 4 : [9 18 16 16 0 0]
Node 5 : [17 0 0 0 0 0]

Minimum Spanning Tree (Kruskal):
Edge (1, 3): weight = 8
Edge (0, 4): weight = 9
Edge (0, 2): weight = 14
Edge (3, 4): weight = 16
Edge (0, 5): weight = 17
Total weight: 64
```

Рисунок 13 — Результат работы алгоритма Крускала

На рисунках 14 и 15 представлен результат выполнения программы для примеров графа «а» с рисунка 1.

```
1  UnorderedGraph gza = new UnorderedGraph(new int[8] [] {
2      new int[8] {0, 19, 0, 0, 0, 25, 0, 6},
3      new int[8] {19, 0, 9, 0, 0, 0, 0, 0},
4      new int[8] {0, 9, 0, 14, 0, 0, 0, 0},
5      new int[8] {0, 0, 14, 0, 21, 2, 0, 0},
6      new int[8] {0, 0, 0, 21, 0, 0, 0, 0},
7      new int[8] {25, 0, 0, 2, 0, 0, 8, 11},
8      new int[8] {0, 0, 0, 0, 0, 8, 0, 17},
9      new int[8] {6, 0, 0, 0, 0, 11, 17, 0}
10 });
11 gza.Print();
12 gza.Kruskal();
```

Рисунок 14 — Граф а

```
Microsoft Visual Studio Debug Console

Graph
Node 0 : [0 19 0 0 0 25 0 6]
Node 1 : [19 0 9 0 0 0 0 0]
Node 2 : [0 9 0 14 0 0 0 0]
Node 3 : [0 0 14 0 21 2 0 0]
Node 4 : [0 0 0 21 0 0 0 0]
Node 5 : [25 0 0 2 0 0 8 11]
Node 6 : [0 0 0 0 0 8 0 17]
Node 7 : [6 0 0 0 0 11 17 0]

Minimum Spanning Tree (Kruskal):
Edge (3, 5): weight = 2
Edge (0, 7): weight = 6
Edge (5, 6): weight = 8
Edge (1, 2): weight = 9
Edge (5, 7): weight = 11
Edge (2, 3): weight = 14
Edge (3, 4): weight = 21
Total weight: 71
```

Рисунок 15 — Граф а

На рисунках 16 и 17 представлен результат выполнения программы для примеров графа «b» с рисунка 1.

```
1 UnorderedGraph gzb = new UnorderedGraph(new int[9] [] {
2     new int[9] {0, 2, 0, 3, 5, 0, 0, 0, 0},
3     new int[9] {2, 0, 10, 0, 0, 0, 0, 0, 0},
4     new int[9] {0, 10, 0, 0, 4, 0, 0, 5, 6},
5     new int[9] {3, 0, 0, 0, 0, 8, 0, 0, 0},
6     new int[9] {5, 0, 2, 0, 0, 7, 0, 0, 0},
7     new int[9] {0, 0, 0, 8, 7, 0, 4, 3, 0},
8     new int[9] {0, 0, 0, 0, 0, 4, 0, 0, 11},
9     new int[9] {0, 0, 5, 0, 0, 3, 0, 0, 0},
10    new int[9] {0, 0, 6, 0, 0, 0, 11, 0, 0}
11 });
12 gzb.Print();
13 gzb.Kruskal();
```

Рисунок 16 — Граф b

```
Microsoft Visual Studio Debug Console

Node 1 : [2 0 10 0 0 0 0 0 0]
Node 2 : [0 10 0 0 4 0 0 5 6]
Node 3 : [3 0 0 0 0 8 0 0 0]
Node 4 : [5 0 2 0 0 7 0 0 0]
Node 5 : [0 0 0 8 7 0 4 3 0]
Node 6 : [0 0 0 0 0 4 0 0 11]
Node 7 : [0 0 5 0 0 3 0 0 0]
Node 8 : [0 0 6 0 0 0 11 0 0]

Minimum Spanning Tree (Kruskal):
Edge (0, 1): weight = 2
Edge (0, 3): weight = 3
Edge (5, 7): weight = 3
Edge (2, 4): weight = 4
Edge (5, 6): weight = 4
Edge (0, 4): weight = 5
Edge (2, 7): weight = 5
Edge (2, 8): weight = 6
Total weight: 32
```

Рисунок 17 — Граф b

4 КОНТРОЛЬНЫЕ ВОПРОСЫ

1) Алгоритм Дейкстры решает задачу поиска кратчайших путей от одной вершины до всех остальных в графе с неотрицательными весами рёбер. Алгоритм Крускала строит минимальное остовное дерево в связном неориентированном взвешенном графе.

2) Алгоритм Дейкстры применим только для графов с неотрицательными весами рёбер. Граф может быть ориентированным или неориентированным, но требуется, чтобы все веса были неотрицательны, иначе результат может быть некорректным.

3) Для реализации поиска в ширину и глубину в C# используются классы для представления графа, а также стандартные коллекции: очередь (`Queue<T>`) для BFS, стек (`Stack<T>`) для DFS (или рекурсивные методы). Эти конструкции позволяют организовать обход вершин в нужном порядке.

4) Сложность алгоритма Дейкстры оценивается в зависимости от способа реализации. При использовании двоичной кучи она составляет $O((V + E)\log V)$, где V — число вершин, E — число рёбер. При простейшей реализации с линейным поиском минимума сложность $O(V^2)$.

5) Алгоритм Дейкстры относится к классу полиномиальных алгоритмов, так как его временная сложность ограничена полиномом от размера входных данных.

6) Точность алгоритма Дейкстры гарантируется при выполнении условий (неотрицательные веса). Её можно проверить на тестовых графах с известными эталонными значениями или опираться на строгое математическое доказательство корректности.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы изучены и реализованы на языке C# основные алгоритмы на графах – алгоритм поиска в ширину и в глубину, алгоритм Дейкстры и алгоритм Крускала.